In God we trust

عنوان پروژه : *زمانبندی در xv6*

دانشجویان : *ریحانه عین‌الهی - بهاره عین‌الهی - مهدیس میرزایی*

شماره پروژه : *پروژه سوم آزمایشگاه سیستم عامل*

لینک گیت‌هاب : *گیت‌هاب پروژه سوم آزمایشگاه*

لینک چت‌بات‌ها: *: لینک چت‌بات‌ها*

دانشکده مهندسی برق و کامپیوتر - دانشگاه تهران

۱۵ آذر ۱۴۰۴

آشنایی عمیق با مفاهیم پایهٔ زمانبندی

پرسش ۱ : ساختار PCB و همچنین وضعیتهای تعریف شده برای هر پردازه را در xv6 پیدا کرده و گزارش کنید. آیا شباهتی میان دادههای موجود در این ساختار و ساختار به تصویر کشیده شده در شکل 3.3 منبع درس وجود دارد؟

پاسخ :

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

Context شامل رجیستر هایی میشود که در هر بار عمل context switch باید ذخیره شوند که با لود شدن دوباره برنامه بتواند روند طبیعی خود را دنبال کند. رجیستر program counter، eip نگهداری میشود.

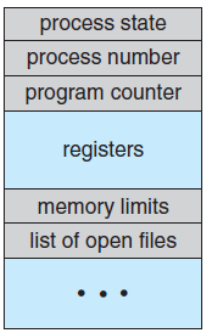


Figure 3.3 Process control block (PCB).

Process state:

```
  enum procstate state;        // Process state
```

Process number:

```
  int pid;                     // Process ID
```

Registers:

```
  struct context *context;     // swtch() here to run process
```

Memory limits:

```
uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this
```

List of open files:

```
  struct file *ofile[NOFILE];  // Open files
```

پرسش ۲ : برای هر وضعیت پردازه در v6× معادل وضعیت مربوطه در شکل ۱ را مشخص کنید.

پاسخ :

استیت های برنامه مشابه توضیحات منبع می باشد.

EMBRYO، پردازه تازه ساخته شده است: new

RUNNABLE، همان ready است و پردازه منتظر cpu است: ready

RUNNING، پردازه ای که در حال حاضر در cpu ران میشود: running

SLEEPING، منتظر یک event است: waiting

ZOMBIE، که وقتی یک پردازه تمام میشود در این استیت قرار میگیرد: terminated

---

پرسش ۳ : با توجه به توضیحات چرخهٔ وضعیت پرداز ها و ساختار xv6، مشخص کنید کدام توابع موجود در proc.c باعث گذار پردازه از حالت new به حالت ready در شکل ۱ میشوند. وضعیت یک پردازه درv6 × در این گذار از چه حالتی/حالتهایی به چه حالت/حالتهایی تغییر میکند؟

پاسخ :

در انتهای تابع userinit که اولین پردازه را تولید میکند، استیت این پردازه به RUNNABLE تغییر میابد:

```
  acquire(&ptable.lock);
  p->state = RUNNABLE;
```

```
  release(&ptable.lock);
```

در حالت کلی باقی پردازه ها که با fork ایجاد میشوند، در این تابع مشابه userinit ابتدا با استفاده از allocproc، پردازه جدیدی در یک خانه allocate میشود:

```
if((np = allocproc()) == 0){
    return -1;
  }
```

و سپس در صورت موفقیت در انتها به RUNNABLE تغییر می یابد:

```
acquire(&ptable.lock);
np->state = RUNNABLE;
release(&ptable.lock);
return pid;
```

پرسش ۴ : ۴) سقف تعداد پردازههای ممکن در xv6 چه عددی است؟ در صورتی که یک پردازه تعداد زیادی پردازهٔ فرزند ایجاد کند و از این سقف عبور کند، کرنل چه واکنشی نشان داده و برنامهٔ سطح کاربر چه بازخوردی دریافت میکند؟

پاسخ :

```
struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;
```

```
#define NPROC        64  // maximum number of processes
```

ماکسیمم تعداد پردازه ها در xv6 64 تعیین شده است.

در صورتی که همه ی خانه های proc شامل مقدار باشند، اسن تابع صفر بر میگرداند و fail میشود:

```
static struct proc* allocproc(void){
  .. .
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == UNUSED)
      goto found;
  release(&ptable.lock);
  return 0;

found:
 .. .
```

و fork نیز fail شده و process جدید ایجاد نمیشود:

```
fork(void){
  int i, pid;
  struct proc *np;
  struct proc *curproc = myproc();
  // Allocate process.
  if((np = allocproc()) == 0){
    return -1;
 }
```

پرسش ۵ :چرا نیاز است در ابتدای هر حلقه تابع scheduler، جدول پردازه ها قفل شود؟ آیا در سیستم هایی که تنها یک پردازنده (یک هستهٔ پردازشی) دارند هم نیاز است این کار صورت بگیرد؟

پاسخ : روی یک ماشین تک‌هسته‌ای هم همزمانی وجود دارد، چون وقفه‌ها در هر لحظه ممکن است فعال شوند و کد کرنل را اجرا کنند که این کد روی جدول پردازش‌ها تغییر ایجاد میکند، بنابراین برای جلوگیری از تغییر نادرست این جدول قبل از شروع خواندن و تغییرات روی جدول آن را lock کنیم.

پرسش ۶ : ۶) با فرض اینکه XV6 در حالت چند پردازنده‌ای (چند هسته‌ای) در حال اجراست، اگر یک پردازه آماده به اجرا شود، و صف پرداز ها در پردازنده دیگری در حال طی شدن باشد (proc:335)، در مکانیزم زمانبندی XV6 نسبت به موقعیت پردازه در صف، در چه iteration ای امکان schedule پیدا میکند؟ (در همان iteration یا در iteration بعدی)

پاسخ :

در همان iteration فعلی زمان‌بندی نمی‌شود در iteration بعدی زمان‌بندی خواهد شد. وقتی یک CPU در تابع scheduler در حال پیمایش process ها است، فقط دارد یک آرایه را از ابتدا تا انتها اسکن می‌کند. اگر در همین زمان یک CPU دیگر وضعیت یک فرایند را به RUNNABLE تغییر دهد، CPU اول از آن خانه‌ی آرایه عبور کرده و در این اسکن متوجه تغییر نخواهد شد.

CPU حلقه را تمام می‌کند، دوباره وارد حلقهٔ بی‌نهایت for می‌شود، و در اسکن کامل بعدی، فرایند را در وضعیت RUNNABLE خواهد دید.

پرسش ۷ : رجیسترهای موجود در ساختار context را نام ببرید.

پاسخ :

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

پرسش ۸ : همانطور که میدانید یکی از مهم ترین رجیسترها قبل از هر تعویض متن Program Counter است که نشان میدهد روند اجرای برنامه تا کجا پیش رفته است. با ذخیره‌سازی این رجیستر می توان محل ادامهٔ برنامه را بازیابی کرد. این رجیستر در ساختار context چه نام دارد؟ شیوه ذخیره و بازنشانی این رجیستر را توضیح دهید.

پاسخ : این رجیستر **eip** است. این رجیستر با تابع **swtch** بازنویسی میشود:

```
// swtch.S
void            swtch(struct context**, struct context*);
```

```
.globl swtch
swtch:
  movl 4(%esp), %eax
  movl 8(%esp), %edx

  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  movl %esp, (%eax)
  movl %edx, %esp

  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret
```

مقادیر به ترتیب در استک قرار میگیرند و سپس از آن در رجیستر ها ذخیره میشوند.

پرسش ۹ : همان‌طور که در قسمت قبل مشاهده کردید، ابتدای تابع scheduler، وقفه به کمک تابع sti، فعال میشود. با توجه به توضیحات این قسمت، اگر وقفه‌ها فعال نمی‌شد چه مشکلی به‌وجود می‌آمد؟ آیا وقفهای در این حالت قابلیت اجرا دارد؟ توضیح دهید.

پاسخ : در حالت عادی، وقتی هیچ پردازه‌ای برای اجرا نیست، CPU وارد scheduler می‌شود و آن را مدام اجرا می‌کند. تابع acquire باعث غیرفعال شدن وقفه‌ها می‌شود که با استفاده از cli این کار را می‌کند. این باعث می‌شود اگر sti نباشد پس از acquire که همه وقفه‌ها را غیرفعال می‌کند تابع release اجرا شود که صرفا به حالت قبلی برمی‌گرداند وقفه‌ها را ولی وقفه‌ها پیش از این هم غیرفعال بودند و این باعث می‌شود به طور کلی وقفه‌ها غیرفعال باشند. حال یکی از وقفه‌ها تایمر است و آن نیز نادیده گرفته می‌شود.

```
case T_IRQ0 + IRQ_TIMER:
    if(cpuid() == 0){
      acquire(&tickslock);
      ticks++;
      wakeup(&ticks);
      release(&tickslock);
    }
```

و طبق شکل بالا چون این باعث می‌شود trick آپدیت نشود و این یعنی هر پردازه‌ای که در حالت sleep باشد در همان حالت باقی می‌ماند و به عبارتی سیستم قفل شود.

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();
  c->proc = 0;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      c->proc = p;
      switchuvm(p);
      p->state = RUNNING;

      swtch(&(c->scheduler), p->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      c->proc = 0;
    }
    release(&ptable.lock);

  }
}
```

پس اگه sti وجود داشته باشد باعث می‌شود قبل (مدت خیلی کمی) و بعد از این دو تابع cpu وقفه‌ها را دریافت کند.

پرسش ۱۰ : وقفهٔ تایمر هر چه مدت یک بار صادر میشود؟ (راهنمایی: می توانید با اضافه کردن یک cprintf پس از ticks++ و یا با مراجعه به تابع lapicinit واقع در lapic.c این موضوع را مشاهده کنید.)

پاسخ : طبق فایل lapic.c و تابع lapicinit مقدار اولیه TICR با 10,000,000 ست شده است و با bus frequency counts down می‌کند (همانطور که در کامنت‌ها نوشته شده) و با سرعت ثابتی کم می‌شود تا به صفر برسد و در qemu این مقدار معادل ۱۰ میلی‌ثانیه است پس با فرکانس ۱۰۰ هرتز در ثانیه عمل می‌کند.

```
// The timer repeatedly counts down at bus frequency
// from lapic[TICR] and then issues an interrupt.
// If xv6 cared more about precise timekeeping,
// TICR would be calibrated using an external time source.
lapicw(TDCR, X1);
lapicw(TIMER, PERIODIC | (T_IRQ0 + IRQ_TIMER));
lapicw(TICR, 10000000);
```

پرسش ۱۱ : با توجه به توضیحات داده شده، چه تابعی منجر به انجام شدن گذار interrupt در شکل ۱ خواهد شد؟

پاسخ : طبق توضیحات گفته شده این کار دقیقا وظیفه تابع yield است و باعث می‌شود از حالت running به runnable تغییر یابد.

```
// Force process to give up CPU on clock tick.
// If interrupts were on while locks held, would need to check nlock.
if(myproc() && myproc()->state == RUNNING &&
   tf->trapno == T_IRQ0+IRQ_TIMER)
  yield();
```

پرسش ۱۲ : با توجه به توضیحات قسمت scheduler dispatch میدانیم زمانبندی در xv6 به شکل نوبت گردشی است. حال با توجه مشاهدات خود در این قسمت، استدلال کنید، کوانتوم زمانی این پیاده‌سازی از زمانبندی نوبت گردشی چند میلی ثانیه است؟

پاسخ : طبق سوال ۱۰ به این نتیجه رسیدیم فرکانس ۱۰۰ هرتز بر ثانیه است یا به عبارتی به ازای هر tick این زمان ۱۰ میلی ثانیه است. این یعنی هر وقفه تایمر ۱۰ میلی‌ثانیه طول می‌کشد. حال باید بدانیم پس از چند tick تابع yield صدا می‌شود که طبق سوال ۱۱ و کد مربوط به آن دقیقا به ازای این حالت tr->trapno باعث می‌شود tick افزایش پیدا کند (کد مربوط به سوال ۹) و در انتها کد این حالت مجدد بررسی می‌شود و صرفا یکبار yield اجرا می‌شود. پس تعداد و زمان را داریم یعنی هر کوانتوم در RR به صورت ۱۰ میلی‌ثانیه است.

پرسش ۱۳ : تابع wait در نهایت از چه تابعی برای منتظر ماندن برای اتمام کار یک پردازه استفاده میکند؟

پاسخ : طبق تابع wait در proc.c و طبق کامنت‌های این کد این کار را تابع sleep انجام می‌دهد و صبر می‌کند تا فرزندان exit شوند.

```
int
wait(void)
{
  struct proc *p;
  int havekids, pid;
  struct proc *curproc = myproc();

  acquire(&ptable.lock);
  for(;;){
    // Scan through table looking for exited children.
    havekids = 0;
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->parent != curproc)
        continue;
      havekids = 1;
      if(p->state == ZOMBIE){
        // Found one.
        pid = p->pid;
        kfree(p->kstack);
        p->kstack = 0;
        freevm(p->pgdir);
        p->pid = 0;
        p->parent = 0;
        p->name[0] = 0;
        p->killed = 0;
        p->state = UNUSED;
        release(&ptable.lock);
        return pid;
      }
    }

    // No point waiting if we don't have any children.
    if(!havekids || curproc->killed){
      release(&ptable.lock);
      return -1;
    }

    // Wait for children to exit.  (See wakeup1 call in proc_exit.)
    sleep(curproc, &ptable.lock);  //DOC: wait-sleep
  }
}
```

پرسش ۱۴ : با توجه به پاسخ سوال قبل، استفاده‌های (دیگر این تابع چیست؟)ذکر یک نمونه کافی است.

پاسخ : از دیگر استفاده‌های این تابع می‌توان به موقعیتی که وقفه باید منتظر یک event باشد مثل وقتی در صف I/O disk است. به عبارتی وقتی می‌خواهد از دیسک چیزی بخواند چون سرعت‌ها متفاوت است تابعی به نام iderw وجود دارد که باعث sleep شدن پردازه تا زمانی که اطلاعاتی که می‌خواهد آماده شود.

```
void
iderw(struct buf *b)
{
  struct buf **pp;

  if(!holdingsleep(&b->lock))
    panic("iderw: buf not locked");
  if((b->flags & (B_VALID|B_DIRTY)) == B_VALID)
    panic("iderw: nothing to do");
  if(b->dev != 0 && !havedisk1)
    panic("iderw: ide disk 1 not present");

  acquire(&idelock);  //DOC:acquire-lock

  // Append b to idequeue.
  b->qnext = 0;
  for(pp=&idequeue; *pp; pp=&(*pp)->qnext)  //DOC:insert-queue
    ;
  *pp = b;

  // Start disk if necessary.
  if(idequeue == b)
    idestart(b);

  // Wait for request to finish.
  while((b->flags & (B_VALID|B_DIRTY)) != B_VALID){
    sleep(b, &idelock);
  }


  release(&idelock);
}
```

پرسش ۱۵ : الف) چه تابعی در سطح کرنل، منجر به آگاهسازی پردازه از رویدادی است که برای آن منتظر بوده است؟ ب) این تابع منجر به گذار از چه وضعیتی به چه وضعیتی در شکل ۱ خواهد شد؟ ج) آیا توابع دیگری وجود دارند که منجر به انجام این گذار شود؟ نام ببرید.

پاسخ : الف) تابع wakeup این وظیفه را دارد. این تابع از تابع کمکی به نام wakeup1 استفاده میکند که این تابع در واقعا منجر به آگاه‌سازی در سطح کرنل می‌شود.

```
// Wake up all processes sleeping on chan.
void
wakeup(void *chan)
{
  acquire(&ptable.lock);
  wakeup1(chan);
  release(&ptable.lock);
}
```

ب) این تابع باعث جابه‌جایی از waiting که sleeping نام دارد به ready که runnable نام دارد برود.

```
static void
wakeup1(void *chan)
{
  struct proc *p;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    if(p->state == SLEEPING && p->chan == chan)
      p->state = RUNNABLE;
}
```

ج) یکی دیگر از این توابع kill است که وقتی یک پردازه در حالت sleeping است و این تابع فراخوانی شود باعث این مدل جابه‌جایی می‌شود. اگر وقتی sleep است kill شود و به استیت runnable منتقل نشود همیشه sleep می‌ماند و این اصلا خوب نیست (همیشه باقی می‌ماند) و باید متوجه شود که kill شده است (با wakeup شدن و رفتن به استیت runnable) به درستی exit شود.

```
int
kill(int pid)
{
  struct proc *p;

  acquire(&ptable.lock);
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->pid == pid){
      p->killed = 1;
      // Wake process from sleep if necessary.
      if(p->state == SLEEPING)
        p->state = RUNNABLE;
      release(&ptable.lock);
      return 0;
    }
  }
  release(&ptable.lock);
  return -1;
}
```

پرسش ۱۶ : در بخش ۳.۳.۲ منبع درس با پردازه‌های Orphan آشنا شدید، رویکرد xv6 در رابطه با این گونه پردازه‌ها چیست؟

پاسخ : در xv6 برای فرزندارن پردازه‌هایی که parent آنها exit شده است به این صورت عمل می‌شود که آنها را kill نمی‌کند بلکه parent ای به آنها اختصاص می‌دهد. این کار در تابع exit انجام می‌شود و آن پردازه init نام دارد که global است و برای آن پردازه‌ها init را به عنوان parent جدیدشان قرار می‌دهیم و اگر آن پردازه زامبی باشد تابع wakeup1 برای initproc صدا زده می‌شود که باعث می‌شود پاک شود و در غیر این صورت شاید هیچ‌وقت آن زامبی کامل حذف نشود.

```
for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
  if(p->parent == curproc){
    p->parent = initproc;
    if(p->state == ZOMBIE)
      wakeup1(initproc);
  }
}
```

زمانبندی هسته‌های ناهمگون

پرسش ۱۷ : ساختار مربوط به CPU در xv6 پیدا کرده و با بررسی فیلدهای آن، گزارش کنید.

پاسخ :

```
struct cpu {
  uchar apicid;                // Local APIC ID
  struct context *scheduler;   // swtch() here to enter scheduler
   struct taskstate ts;              // Used by x86 to find stack for
interrupt
  struct segdesc gdt[NSEGS];   // x86 global descriptor table
  volatile uint started;       // Has the CPU started?
  int ncli;                    // Depth of pushcli nesting.
   int intena;                      // Were interrupts enabled before
pushcli?
   struct proc *proc;           // The process running on this cpu or
null
};
```

فیلد **apicid** برای کرنل است که هنگام هندل کردن یک اینتراپت مشخص شود برای این cpu هندل و انجام میشود.

فیلد **scheduler** برای وقتی است که یک پردازه exit یا yield شد، context swtich با scheduler اتفاق بیفتد و process بعدی، schedule شود.

فیلد `ts` برای ذخیره stack pointer کرنل است که هنگام انتقال از user mode به kernel mode نیاز میشود.

فیلد `gdt` بخش‌های حافظه (کد، داده، پشته) مورد استفاده هسته و برنامه‌های کاربر را تعریف می‌کند. هر CPU یک GDT جداگانه دارد تا مدیریت بخش‌بندی حافظه به‌طور مستقل انجام شود.

فیلد `started` مشخص میکند که آیا این پردازنده با موفقیت به اجرای کرنل به ویژه scheduler رسیده است یا نه.

فیلد `ncli` نشان میدهد چند اینتراپت غیرفعال شده اند و با pushcli ذخیره شدند تا بعدا با popcli دوباره فعال شوند.

فیلد `proc` یک پوینتر به پردازه ای است که در حال حاضر روی پردازنده در حال اجرا است.

1.4 هسته های E با شماره CPUID زوج: زمانبند نوبت گردشی با کوانتوم زمانی

makefile:

```
CPUS := 1
```

Param.h:

```
#define TIME_QUANTUM 3
```

Proc.c:

To implement the circular queue and track the last process in the array to continue from there:

```
int last_process = -1;
```

```
for(int i =last_selected_process + 1; (i%NPROC)!= last_selected_process; i++){
      if(ptable.proc[i].state != RUNNABLE)
        continue;

      p = &ptable.proc[i%NPROC];
      last_selected_process = i%NPROC;
```

Proc.h:

```
struct proc {
  ...
  int ticks_run;
};
```

Trap.c:

```
trap(struct trapframe *tf)
{
  ...
  switch(tf->trapno){
  case T_IRQ0 + IRQ_TIMER:
    if(cpuid() == 0){
      ...
    }
    if(myproc() != 0){
      myproc()->ticks_run++;

       cprintf("Tick:   PID   %d,   ticks_run   %d\n",   myproc()->pid,
myproc()->ticks_run);

      if(myproc()->ticks_run >= TIME_QUANTUM){
          myproc()->ticks_run = 0;
          myproc()->state = RUNNABLE;
          yield();
      }
    }
```

Test Program, 4_1_test:

```
int main() {
   volatile int i;
   volatile int sum = 0;
   for(i = 0; i < 5; i++){
       if(fork() == 0){
           volatile int j;
           for(j=0;j<10000000;j++)
                   sum+=j;
           exit();
       }
   }
   for(i=0;i<5;i++) {
```

```
        int pid = wait();
        printf(1, "Child %d done\n", pid);
    }
    exit();
}
```

Result:

```
$ 4_1_test
Tick: PID 4, ticks_run 1, ticks =383
Tick: PID 4, ticks_run 2, ticks =384
Tick: PID 4, ticks_run 3, ticks =385
YIELD: PID 4,
Tick: PID 5, ticks_run 1, ticks =386
Tick: PID 5, ticks_run 2, ticks =387
Tick: PID 5, ticks_run 3, ticks =388
YIELD: PID 5,
Tick: PID 6, ticks_run 1, ticks =389
Tick: PID 6, ticks_run 2, ticks =390
Tick: PID 6, ticks_run 3, ticks =391
YIELD: PID 6,
Tick: PID 7, ticks_run 1, ticks =392
Tick: PID 7, ticks_run 2, ticks =393
Tick: PID 7, ticks_run 3, ticks =394
YIELD: PID 7,
Tick: PID 8, ticks_run 1, ticks =396
Tick: PID 8, ticks_run 2, ticks =397
Tick: PID 8, ticks_run 3, ticks =398
YIELD: PID 8,
Tick: PID 4, ticks_run 1, ticks =399
Tick: PID 4, ticks_run 2, ticks =400
Tick: PID 4, ticks_run 3, ticks =401
YIELD: PID 4,
Tick: PID 5, ticks_run 1, ticks =402
Tick: PID 5, ticks_run 2, ticks =403
Tick: PID 5, ticks_run 3, ticks =404
YIELD: PID 5,
Tick: PID 6, ticks_run 1, ticks =405
Tick: PID 6, ticks_run 2, ticks =406
Tick: PID 6, ticks_run 3, ticks =407
YIELD: PID 6,
Tick: PID 7, ticks_run 1, ticks =408
Tick: PID 7, ticks_run 2, ticks =409
Tick: PID 7, ticks_run 3, ticks =410
YIELD: PID 7,
Tick: PID 8, ticks_run 1, ticks =411
Tick: PID 8, ticks_run 2, ticks =412
Tick: PID 8, ticks_run 3, ticks =413
YIELD: PID 8,
Tick: PID 4, ticks_run 1, ticks =414
Tick: PID 4, ticks_run 2, ticks =415
Tick: PID 5, ticks_run 1, ticks =416
Tick: PID 5, ticks_run 2, ticks =417
Tick: PID 5, ticks_run 3, ticks =418
YIELD: PID 5,
Tick: PID 7, ticks_run 1, ticks =419
```

```
Tick: PID 8, ticks_run 1, ticks =420
Tick: PID 8, ticks_run 2, ticks =421
Tick: PID 8, ticks_run 3, ticks =422
YIELD: PID 8,
Tick: PID 3, ticks_run 1, ticks =423
Tick: PID 3, ticks_run 2, ticks =424
Child 4 done
Child 6 done
Child 7 done
CTick: PID 3, ticks_run 1, ticks =425
hild 5 done
Child 8 done
```

2.4 جدا کردن صف هر هسته

در این قسمت باید به نحوی تغییر دهیم که تمامی هسته‌ها صف خودشان را داشته باشند. به عبارتی در scheduler به جای اینکه روی تمامی پردازه‌های iteration انجام دهیم باید روی صف هر کدام این کار را انجام دهیم. ابتدا به ازای هر پردازه باید پردازه next آن را داشته باشیم.

```
  char name[16];               // Process name (debugging)
  uint creation_time;
  int cpu_affinity;
  uint arrival_time;
  struct proc *next;
};
```

سپس باید به ازای هر کدام از پردازه‌ها صف آن را داشته باشیم.

```
struct runq {
  struct spinlock lock;
  struct proc *head;
};

struct runq runqueues[NCPU];

void
pinit(void)
{
  initlock(&ptable.lock, "ptable");
  char name[16];
  for(int i=0; i<NCPU; i++){
    safestrcpy(name, "runq_", 6);
    name[5] = i + '0';
    name[6] = 0;
    initlock(&runqueues[i].lock, name);
    runqueues[i].head = 0;
  }
}
```

و برای اضافه یا کم کردن از صف نیز دو تابع زیر را تعریف میکنیم.

```
void
enqueue_cpu(int cpu_id, struct proc *p)
{
  acquire(&runqueues[cpu_id].lock);
  p->cpu_affinity = cpu_id;
  p->next = 0;
  if(runqueues[cpu_id].head == 0){
    runqueues[cpu_id].head = p;
  } else {
    struct proc *curr = runqueues[cpu_id].head;
    while(curr->next != 0)
      curr = curr->next;
    curr->next = p;
  }
  release(&runqueues[cpu_id].lock);
}

int
dequeue_cpu(int cpu_id, struct proc *p)
{
  acquire(&runqueues[cpu_id].lock);
  struct proc *curr = runqueues[cpu_id].head;
  struct proc *prev = 0;
  while(curr){
    if(curr == p){
      if(prev) prev->next = curr->next;
      else runqueues[cpu_id].head = curr->next;
      curr->next = 0;
      release(&runqueues[cpu_id].lock);
      return 0;
    }
    prev = curr;
    curr = curr->next;
  }
  release(&runqueues[cpu_id].lock);
  return -1;
}
```

و در ابتدا باید init را در cpu 0 شروع کنیم پس

```
  acquire(&ptable.lock);

  p->state = RUNNABLE;
  p->arrival_time = ticks;
  release(&ptable.lock);
  enqueue_cpu(0, p);
}
```

و در توابع fork و yield و wakeup1 و kill نیز تغییرات را اعمال میکنیم.

```
  release(&ptable.lock);

  int target_cpu = find_lightest_e_core();
  enqueue_cpu(target_cpu, np);

  return pid;
}
```

```
void
yield(void)
{
  acquire(&ptable.lock);
  myproc()->state = RUNNABLE;
  myproc()->arrival_time = ticks;
  release(&ptable.lock);
  enqueue_cpu(cpuid(), myproc());
  acquire(&ptable.lock);
  sched();
  release(&ptable.lock);
}
```

```
if(p->state == SLEEPING) {
        p->state = RUNNABLE;
        p->arrival_time = ticks;
        enqueue_cpu(p->cpu_affinity, p);
      }
```

```
static void
wakeup1(void *chan)
{
  struct proc *p;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
    if(p->state == SLEEPING && p->chan == chan) {
      p->state = RUNNABLE;
      p->arrival_time = ticks;
      enqueue_cpu(p->cpu_affinity, p);
    }
  }
}
```

از طرفی دو تابع scheduler و balance load نیز تغییر می‌کنند. در تابع scheduler صرفا صف مربوط به پردازه قفل می‌شود و این باعث ساده شدن دو الگوریتم می‌شود.

```
void
scheduler(void)
{
  struct cpu *c = mycpu();
  struct proc *p;
  c->proc = 0;
  for(;;){
    sti();
    balance_load(c);
    acquire(&runqueues[c->apicid].lock);
    struct proc *head = runqueues[c->apicid].head;
    if(!head){
      release(&runqueues[c->apicid].lock);
      continue;
    }
    struct proc *chosen = 0;
    if(c->apicid % 2 == 0) {
      chosen = select_process_rr(head);
    } else {
      chosen = select_process_fcfs(head);
    }
    if(chosen) {
      detach_process_from_queue(c, chosen, head);
      release(&runqueues[c->apicid].lock);
      execute_process(c, chosen);
    } else {
      release(&runqueues[c->apicid].lock);
    }
  }
}
```

```
extern uint ticks;
void
balance_load(struct cpu *c)
{
  if(!should_attempt_balance(c))
    return;

  c->last_lb = ticks;

  int my_load = get_cpu_load(c->apicid);
  int target_p_load = 0;
  int target_p_cpu = find_lightest_p_core(&target_p_load);

  if(target_p_cpu == -1)
    return;

  if(my_load >= target_p_load + 3) {
    acquire(&runqueues[c->apicid].lock);

    struct proc *victim_prev = 0;
    struct proc *victim = find_migration_victim(runqueues[c->apicid].head,
&victim_prev);

    if(victim != 0) {
      perform_migration(c, target_p_cpu, victim, victim_prev);
    } else {
      release(&runqueues[c->apicid].lock);
    }
  }
}
```

در تابع load balance نیز در قسمتی که می‌خواهیم پردازه را جابه‌جا کنیم تغییراتی داریم که با استفاده از arrival_time و صف مربوط به هر یک از هسته‌ها دنبال اولین پردازه‌ای که وارد صف شده است هستیم و دلیل استفاده از arrival_time این است که اگر از head آن صف استفاده کنیم باعث می‌شود باز هم چک کنیم sh یا init نباشد و باز هم iterate می‌کنیم و از طرفی استفاده از head ممکن است از همان پردازه‌ای که همین لحظه وارد صف شده است یا وضعیت ناپایداری دارد استفاده شود و این arrival_time صرفا باعث مطمئن شدن از درستی اینکه این پردازه از پردازه‌های خاص نیست و زودتر از سایرین وارد صف شده است و پیدا کردن آن در تابع find_migration_victim آمده است.

```
struct proc*
find_migration_victim(struct proc *head, struct proc **prev_out)
{
  struct proc *curr = head;
  struct proc *prev = 0;
  struct proc *victim = 0;
  struct proc *victim_prev = 0;

  while(curr != 0) {
    if(curr->pid > 2 && !is_shell_proc(curr)) {
      if(victim == 0 || curr->arrival_time < victim->arrival_time) {
        victim = curr;
        victim_prev = prev;
      }
    }
    prev = curr;
    curr = curr->next;
  }

  if (prev_out) *prev_out = victim_prev;
  return victim;
}
```

و سپس برای این جابه‌جایی از تابع زیر استفاده می‌شود.

```
void
perform_migration(struct cpu *c, int target_cpu, struct proc *victim, struct proc
*victim_prev)
{
  if(victim_prev == 0) {
    runqueues[c->apicid].head = victim->next;
  } else {
    victim_prev->next = victim->next;
  }
  victim->next = 0;

  release(&runqueues[c->apicid].lock);

  enqueue_cpu(target_cpu, victim);
}
```

سپس در scheduler فرآیند مانند قبل انجام می‌شود با این تفاوت که ابتدا صف مربوط به هسته را قفل می‌کنیم و چک می‌کنیم اگر خالی باشد قفل را آزاد و فرایند خاتمه یابد و اگر خالی نبود با توجه به زوج یا فرد بودن الگوریتم‌ها را اجرا می‌کنیم و حال چون صف داریم این فرایند خیلی ساده است.

```
struct proc*
select_process_rr(struct proc *head)
{
  return head;
}

struct proc*
select_process_fcfs(struct proc *head)
{
  struct proc *curr = head;
  struct proc *chosen = head;
  while(curr){
    if(curr->creation_time < chosen->creation_time)
      chosen = curr;
    curr = curr->next;
  }
  return chosen;
}
```

و سپس اگر پردازه‌ای بود آن را از صف پردازه‌های آن هسته خارج می‌کنیم.

```
void
detach_process_from_queue(struct cpu *c, struct proc *chosen, struct proc *head)
{
  if(chosen == head) {
    runqueues[c->apicid].head = chosen->next;
  } else {
    struct proc *tmp = head;
    while(tmp->next != chosen)
      tmp = tmp->next;
    tmp->next = chosen->next;
  }
  chosen->next = 0;
}
```

و قفل را آزاد می‌کینم و آن پردازه را execute می‌کنیم.

```
void
execute_process(struct cpu *c, struct proc *p)
{
  c->proc = p;
  acquire(&ptable.lock);
  switchuvm(p);
  p->state = RUNNING;
  swtch(&(c->scheduler), p->context);
  switchkvm();
  c->proc = 0;
  release(&ptable.lock);
}
```

و برای تست این قسمت برنامه زیر را در نظر گرفتیم.

```
#include "types.h"
#include "stat.h"
#include "user.h"

void spin(int id) {
  int i;
  int j;
  for(i = 0; i < 10000000; i++) {
    j = i * i;
  }
  printf(1, "Child %d (on CPU ?) finished.\n", id);
}
int main(int argc, char *argv[]) {
  int i;
  int children = 6;
  printf(1, "Starting Queue Isolation Test...\n");
  for(i = 0; i < children; i++) {
    int pid = fork();
    if(pid == 0) {
      spin(getpid());
      exit();
    }
  }
  for(i = 0; i < children; i++) {
    wait();
  }
  printf(1, "Test finished.\n");
  exit();
}
```

و خروجی به صورت زیر شد

```
Booting from Hard Disk...
cpu1: starting 1
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ queuetest
Starting Queue Isolation Test...
Child 4 (on CPU ?) fChild 6 (on CPU ?) fiChild 8inishedCnhi lChdi sh5C .(oen(
CPUo d?h)n iflin.i
s hedd.
7 ild 9 (C(PonU C ?P)U  fionn i?sChPedU.)
 ? fini) finished.
shed.
Test finished.
$
```

3.4 هسته های P با شماره CPUID فرد : زمانبند اولین ورود رسیدگی

در این قسمت برای پیاده‌سازی FCFS باید یک فیلد به عنوان creation_time به هر کدام از process ها اختصاص دهیم تا بر اساس آن بتوان متوجه شد چه زمانی وارد شده است. همچنین باید در scheduler تغییراتی ایجاد کنیم که بر اساس process هایی که در استیت runnable هستند آن موردی که creation_time کمتری دارد را انتخاب کنیم. پس به صورت زیر خواهد بود :

در فایل proc.h صرفا این فیلد creation_time را اضافه میکنیم.

```
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  uint creation_time;
};
```

و در فایل proc.c در دو قسمت تغییراتی ایجاد می‌کنیم. هنگام allocproc در process این مقدار را با توجه به tick مقداردهی می‌کنیم. در قسمت scheduler نیز صرفا تغییراتی ایجاد میکنیم. در این حالت صرفا با توجه به cpuid به دو گروه زوج و فرد تقسیم می‌شود. وقتی فرد باشد باید fcfs اجرا شود. حال در این حالت باید بین تمامی process هایی که استیت آنها runnable است مقایسه انجام دهیم و آن process ای که creation_time کمتری دارد از بین تمامی موارد پیدا کنیم. پیش از این تمامی process ها با هر نوع cpuid یک روند داشتند ولی حال باید با توجه به cpuid تقسیم‌بندی کنیم.

```
if(cpuid() % 2 != 0){
      struct proc *min_p;
      min_p = 0;
      for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->state != RUNNABLE)
          continue;
        if(min_p == 0 || p->creation_time < min_p->creation_time){
          min_p = p;
        }
      }

      if(min_p != 0){
        p = min_p;
        c->proc = p;
        switchuvm(p);
        p->state = RUNNING;
        swtch(&(c->scheduler), p->context);
        switchkvm();
        c->proc = 0;
      }
    }
```

```
found:
  p->state = EMBRYO;
  p->pid = nextpid++;
  p->creation_time = ticks;
```

همچنین برای تست برنامه یک فایل جدید ایجاد و در makefile اضافه کردیم.

```
#include "types.h"
#include "stat.h"
#include "user.h"

void spin_cpu() {
  int i;
  int j;
  for (i = 0; i < 5000000; i++) {
    j = i * i;
  }
}

int main(int argc, char *argv[]) {
  int n_children = 5;
  int pid;

  printf(1, "Starting FCFS Creation Time Test...\n");

  for (int i = 0; i < n_children; i++) {
    pid = fork();
    if (pid < 0) {
      printf(1, "Fork failed\n");
    } else if (pid == 0) {
      spin_cpu();
      printf(1, "Child %d (Created #%d) finished execution.\n", getpid(), i + 1);
      exit();
    } else {
      sleep(10);
    }
  }

  for (int i = 0; i < n_children; i++) {
    wait();
  }

  printf(1, "Test finished.\n");
  exit();
}
```

و خروجی به صورت زیر شد :

```
init: starting sh
$ test_fcfs
Starting FCFS Creation Time Test...
Child 4 (Created #1) finished execution.
Child 5 (Created #2) finished execution.
Child 6 (Created #3) finished execution.
Child 7 (Created #4) finished execution.
Child 8 (Created #5) finished execution.
Test finished.
```

4.4 جابه‌جایی پردازه میان صف‌ها

هدف این قسمت جابه‌جایی پردازه میان صف‌ها است و انتقال از زوج به فرد است. برای این هدف فیلدی به proc اضافه می‌شود به نام cpu_affinity که در واقع برای این است که بدانیم یک پردازه روی کدام هسته است.

```
  char name[16];               // Process name (debugging)
  uint creation_time;
  int cpu_affinity;
};
```

و در fork و allocproc و userinit این فیلد مقداردهی می‌شود که صرفا در fork این مقدار برابر با هسته E با کمترین تعداد پردازه است و سایرین 0 است.

```
void
userinit(void)
{
  struct proc *p;
  extern char _binary_initcode_start[], _binary_initcode_size[];

  p = allocproc();
  p->cpu_affinity = 0;
```

```
found:
  p->state = EMBRYO;
  p->pid = nextpid++;
  p->creation_time = ticks;
  p->cpu_affinity = 0;
```

```
  np->cpu_affinity = find_lightest_e_core();
```

و تابعی که E با کمترین تعداد پردازه را پیدا می‌کند به این صورت است :

```
int
find_lightest_e_core(void)
{
  int min_load = 100000;
  int target_id = 0; // Default to CPU 0 if all else fails

  for(int i = 0; i < ncpu; i++){
    if(cpus[i].apicid % 2 == 0) { // Even = E-core
      int current_load = get_cpu_load(cpus[i].apicid);
      if(current_load < min_load){
        min_load = current_load;
        target_id = cpus[i].apicid;
      }
    }
  }
  return target_id;
}
```

سپس در تابع scheduler قبل از اینکه با توجه به هسته RR یا FCFS اجرا شود, بررسی می‌کنیم آیا شرایط جابه‌جایی هست یا نه ؟ این کار توسط تابع balance_load انجام می‌شود و صرفا برای زوج‌ها است.

```
  for(;;){
    sti();
    acquire(&ptable.lock);
    balance_load(c);
```

```
extern uint ticks;

void
balance_load(struct cpu *c)
{
  if(c->apicid % 2 != 0) return;
  if((ticks - c->last_lb) < 5) return;

  c->last_lb = ticks;

  int my_load = get_cpu_load(c->apicid);
  int target_p_load = 0;
  int target_p_cpu = find_lightest_p_core(&target_p_load);

  if(target_p_cpu == -1) return;

  if(my_load > 1) {
    cprintf("LB CHECK: E-Core%d(L=%d) vs P-Core%d(L=%d)\n", c->apicid, my_load,
target_p_cpu, target_p_load);
  }

  if(my_load >= (target_p_load + 3)) {
      struct proc *victim = find_victim_proc(c->apicid);

      if(victim != 0){
        cprintf("LoadBalance: Moving PID %d from CPU%d to CPU%d\n", victim->pid, c-
>apicid, target_p_cpu);
        victim->cpu_affinity = target_p_cpu;
      }
  }
}
```

در این تابع به این صورت است که ابتدا زوج بودن را بررسی می‌کند. همچنین چون صرفا هر ۵۰ میلی‌ثانیه باید بررسی شود پس اگر قبل از این زمان وارد این تابع شویم درجا return می‌شود. این مقدار که دفعه قبل چه زمانی در تابع بودیم توسط فیلدی که به برای cpu تعریف شده است و ticks که بر اساس وقفه تایمر آپدیت می‌شود مقایسه می‌شود و اگر از دفعه قبل تا این لحظه ۵ تیک گذشته باشد (طبق پرسش ۱۰ میدانیم این مقدار ۱۰ میلی‌ثانیه است) یعنی معادل ۵۰ میلی‌ثانیه می‌شود و اگر کمتر باشد از تابع خارج می‌شویم.

```
  struct proc *proc;           // The process running on this cpu or null
  uint last_lb;
};
```

حال با استفاده از تابع get_cpu_load تعداد پردازه‌های این cpu زوج را به دست می‌آوریم.

```
int
get_cpu_load(int apicid)
{
  struct proc *p;
  int count = 0;
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state == RUNNABLE && p->cpu_affinity == apicid) {
      count++;
    }
  }
  return count;
}
```

و با تابع find_lightest_p_core از بین همه فردها آن هسته با کمترین تعداد پردازه به دست می‌آوریم.

```
int
find_lightest_p_core(int *min_load_out)
{
  int min_load = 100000;
  int target_id = -1;

  for(int i = 0; i < ncpu; i++){
    if(cpus[i].apicid % 2 != 0) {
      int current_load = get_cpu_load(cpus[i].apicid);
      if(current_load < min_load){
        min_load = current_load;
        target_id = cpus[i].apicid;
      }
    }
  }
  *min_load_out = min_load;
  return target_id;
}
```

سپس بررسی می‌شود آیا حداقل سه‌تا بیشتر است و اگر بود با استفاده از تابع find_victim_proc بررسی می‌شود از بین پردازه‌های در استیت runnable که مربوط به این هسته هستند آیا پردازه‌های خاص sh یا init نباشند که با تابع is_shell_proc چک می‌شود و از بین سایرین arrival_time کمتری داشته باشد انتخاب می‌شود. این متغیر یکی دیگر از فیلدهای اضافه شده به proc است و هنگامیکه یک process به استیت runnable می‌رود یا به عبارتی در صف است مقداردهی می‌شود.

```
struct proc* find_victim_proc(int source_apicid)
{
  struct proc *p;
  struct proc *victim = 0;

  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->state == RUNNABLE && p->cpu_affinity == source_apicid){
      if(p->pid > 2 && !is_shell_proc(p)){
        if(victim == 0 || p->arrival_time < victim->arrival_time){
          victim = p;
        }
      }
    }
  }
  return victim;
}
```

```
int
is_shell_proc(struct proc *p)
{
  if(p->name[0] == 's' && p->name[1] == 'h' && p->name[2] == 0) return 1;
  return 0;
}
```

و یک فایل برای تست این فرایند قرار دادیم .

```
#include "types.h"
#include "stat.h"
#include "user.h"

int main(int argc, char *argv[]) {
  int n_children = 8;
  int pid;

  printf(1, "--- Starting Load Balancing Test ---\n");

  for (int i = 0; i < n_children; i++) {
    pid = fork();
    if (pid == 0) {
      volatile int x = 0;
      int start_time = uptime();
      while (uptime() - start_time < 200) {
        for(int k=0; k<10000; k++) {
          x = x + 1;
        }
      }
      printf(1, "Child %d Done\n", getpid());
      exit();
    }
  }

  volatile int y = 0;
  int start_time = uptime();
  while (uptime() - start_time < 200) {
    for(int k=0; k<10000; k++) {
      y = y + 1;
    }
  }

  for (int i = 0; i < n_children; i++) {
    wait();
  }

  printf(1, "--- Test Finished ---\n");
  exit();
}
```

و خروجی به صورت زیر است :

```
$ test_lb
--- Starting Load Balancing Test ---
LB CHECK: E-Core0(L=5) vs P-Core1(L=0)
LoadBalance: Moving PID 5 from CPU0 to CPU1
LB CHECK: E-Core0(L=8) vs P-Core1(L=0)
LoadBalance: Moving PID 6 from CPU0 to CPU1
LB CHECK: E-Core0(L=7) vs P-Core1(L=1)
LoadBalance: Moving PID 7 from CPU0 to CPU1
LB CHECK: E-Core0(L=6) vs P-Core1(L=2)
LoadBalance: Moving PID 8 from CPU0 to CPU1
LB CHECK: E-Core0(L=5) vs P-Core1(L=3)
```

5.4 ارزیابی الگوریتم های زمان بندی

Syscall.h:

```
#define SYS_start  22
#define SYS_end    23
```

usys.S:

```
SYSCALL(start)
SYSCALL(end)
```

Syscall.c:

```
extern int sys_start(void);
extern int sys_end(void);
[SYS_start]   sys_start,
[SYS_end]   sys_end,
```

Proc.c:

```
int measuring = 0;
uint start_tick;
```

```
Void exit(void){
 if (measuring)
   myproc()->completed_in_interval = 1;
}
```

Start system call is used to initialise counting:

```
int sys_start(void) {
   acquire(&tickslock);
   start_tick = ticks;
   release(&tickslock);
   measuring = 1;
   struct proc *p;
   for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
       p->completed_in_interval = 0;
   }
   return 0;
}
```

End System Call stops the tracking and print the result from start till now:

```
int sys_end(void) {
   if (!measuring) {
       cprintf("Error: Measurement not started.\n");
       return -1;
   }
   measuring = 0;
   acquire(&tickslock);
   uint end_tick = ticks;
   release(&tickslock);
   uint total_ticks = end_tick - start_tick;
   int completed_count = 0;
   struct proc *p;
   for (p = ptable.proc; p < &ptable.proc[NPROC]; p++)
       if (p->completed_in_interval)
           completed_count++;
   cprintf("--- Throughput Measurement ---\n");
   cprintf("Total Ticks: %d\n", total_ticks);
   cprintf("Completed Processes: %d\n", completed_count);

   if (total_ticks > 0) {
     cprintf("Throughput (Completed/Tick): %d / %d\n",
             completed_count, total_ticks);
    else
     cprintf("Time interval too short (0 ticks).\n");
   return 0;
}
```

2 .5.4 فراخوانی اطلاعات پردازه ها

```
int
sys_psinfo(void)
{
 struct proc *p;
 cprintf("\n--- Process Information ---\n");
 cprintf("---------------------------------------\n");
 acquire(&ptable.lock);
 for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
   if (p->state == UNUSED)
     continue;
   acquire(&tickslock);
   uint current_tick = ticks;
   release(&tickslock);

   uint elapsed_ticks = 0;
   if (p->creation_time > 0 && current_tick >= p->creation_time) {
     elapsed_ticks = current_tick - p->creation_time;
   }

    cprintf("PID: %d STATUS: %s PARENT: %d NAME: %s ELAPSED(ticks): %d
CPU: %d Algorithm: %s\n",
     p->pid,
     state_to_str(p->state),
     (p->parent) ? p->parent->pid : 0,
     p->name,
     elapsed_ticks,
     p->last_cpuid,
        (p->last_cpuid == -1)?"didn't run": p->last_cpuid%2? "FCFS" :
"RR");

 }
 release(&ptable.lock);
 cprintf("---------------------------------------\n\n");
  return 0;
}
```

6.4 برنامه های سطح کاربر

در این بخش برنامه ای نوشته شده است تا قابلیت های اضافه شده در این پروژه بررسی شوند:

1. الگوریتم round robin برای cpu های زوج
2. الگوریتم FCFS برای cpu های فرد
3. جدا شدن صف پردازه ها
4. Balance کردن لود cpu ها
5. سیستم کال های start و end و psinfo

این خروجی به ازای هر tick از زمان است که به درک دقیق فرآیند های cpu ها کمک میکند.

تابع psinfo در هر iteration صدا زده شده است تا اطلاعات پردازه ها را نمایش دهد.

همین طور که در خط 27 تا 37 مشاهده میشود هر پردازه حداکثر 3 tick در cpu وجود دارد و بعد از آن yield میشود تا پردازه ی بعدی جای آن را بگیرد.

جابه جایی در صف ها در 52 دیده میشود که process 5 که چهارمین پردازه cpu زوج است به فرد منتقل شده است.

همین طور که از خط 60 پردازه 5 در cpu فرد قرار میگیرد طبق FCFS به طور کامل انجام میشود، سپس به پردازه بعدی وارد شده به صف میرود.

در انتهای troughtput که با تابع end باید نمایش داده میشد آمده است.

```
int main() {
   start();
   volatile int i;
   volatile int sum = 0;
   for(i = 0; i < 3; i++){
       psinfo();
       if(fork() == 0){
           volatile int j;
           for(j=0;j<10000000;j++)
                   sum+=j;
           exit();
       }
   }
   for(i=0;i<3;i++)
       int pid = wait();
   end();
   exit();
}
```

```
1  4_6_test
2  Tick: PID 3,cpu : Even ,ticks_run 1, ticks =599
```

```
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =600
Tick: PID 3,cpu : Even ,ticks_run 3, ticks =601
YIELD: PID 3,

Tick: PID 3,cpu : Even ,ticks_run 1, ticks =602
--- Process Information ---
-------------------------------------
PID: 1 STATUS: SLEEPING  PARENT: 0 NAME: init ELAPSED(ticks): 0 CPU: 0 Algorithm: RR
PID: 2 STATUS: SLEEPING  PARENT: 1 NAME: sh ELAPSED(ticks): 558 CPU: 0 Algorithm: RR
PID: 3 STATUS: RUNNING   PARENT: 2 NAME: 4_6_test ELAPSED(ticks): 8 CPU: 0 Algorithm: RR
-------------------------------------
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =603
Tick: PID 3,cpu : Even ,ticks_run 3, ticks =604
YIELD: PID 3,


--- Process Information ---
-------------------------------------
PID: 1 STATUS: SLEEPING  PARENT: 0 NAME: init ELAPSED(ticks): 0 CPU: 0 Algorithm: RR
PID: 2 STATUS: SLEEPING  PARENT: 1 NAME: sh ELAPSED(ticks): 561 CPU: 0 Algorithm: RR
PID: 3 STATUS: RUNNING   PARENT: 2 NAME: 4_6_test ELAPSED(ticks): 11 CPU: 0 Algorithm: RR
PID: 4 STATUS: RUNNABLE  PARENT: 3 NAME: 4_6_test ELAPSED(ticks): 0 CPU: 0 Algorithm: RR
-------------------------------------

Tick: PID 3,cpu : Even ,ticks_run 1, ticks =606
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =607
Tick: PID 3,cpu : Even ,ticks_run 3, ticks =608
YIELD: PID 3,
Tick: PID 4,cpu : Even ,ticks_run 1, ticks =610
Tick: PID 4,cpu : Even ,ticks_run 2, ticks =611
Tick: PID 4,cpu : Even ,ticks_run 3, ticks =612
YIELD: PID 4,

Tick: PID 3,cpu : Even ,ticks_run 1, ticks =613
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =614
--- Process Information ---
-------------------------------------
PID: 1 STATUS: SLEEPING  PARENT: 0 NAME: init ELAPSED(ticks): 0 CPU: 0 Algorithm: RR
PID: 2 STATUS: SLEEPING  PARENT: 1 NAME: sh ELAPSED(ticks): 570 CPU: 0 Algorithm: RR
PID: 3 STATUS: RUNNING   PARENT: 2 NAME: 4_6_test ELAPSED(ticks): 20 CPU: 0 Algorithm: RR
PID: 4 STATUS: RUNNABLE  PARENT: 3 NAME: 4_6_test ELAPSED(ticks): 9 CPU: 0 Algorithm: RR
PID: 5 STATUS: RUNNABLE  PARENT: 3 NAME: 4_6_test ELAPSED(ticks): 8 CPU: 0 Algorithm: RR
-------------------------------------
Tick: PID 3,cpu : Even ,ticks_run 3, ticks =615
YIELD: PID 3,
Tick: PID 4,cpu : Even ,ticks_run 1, ticks =617
Tick: PID 4,cpu : Even ,ticks_run 2, ticks =618
Tick: PID 4,cpu : Even ,ticks_run 3, ticks =619
YIELD: PID 4,
process with ID: 5, migrated to CPU: 1

Tick: PID 3,cpu : Even ,ticks_run 1, ticks =621
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =622
Tick: PID 5,cpu : Odd ,ticks_run 1, ticks =622
Tick: PID 5,cpu : Odd ,ticks_run 2, ticks =622
Tick: PID 4,cpu : Even ,ticks_run 1, ticks =623
Tick: PID 4,cpu : Even ,ticks_run 2, ticks =624
Tick: PID 5,cpu : Odd ,ticks_run 3, ticks =624
Tick: PID 4,cpu : Even ,ticks_run 3, ticks =625
YIELD: PID 4,
Tick: PID 5,cpu : Odd ,ticks_run 4, ticks =626
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =627
Tick: PID 5,cpu : Odd ,ticks_run 5, ticks =627
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =628
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =629
YIELD: PID 6,
Tick: PID 5,cpu : Odd ,ticks_run 6, ticks =628
Tick: PID 5,cpu : Odd ,ticks_run 7, ticks =629
Tick: PID 4,cpu : Even ,ticks_run 1, ticks =630
Tick: PID 5,cpu : Odd ,ticks_run 8, ticks =630
Tick: PID 4,cpu : Even ,ticks_run 2, ticks =631
Tick: PID 4,cpu : Even ,ticks_run 3, ticks =632
YIELD: PID 4,
```

```
Tick: PID 5,cpu : Odd ,ticks_run 9, ticks =632
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =633
Tick: PID 5,cpu : Odd ,ticks_run 10, ticks =633
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =634
Tick: PID 5,cpu : Odd ,ticks_run 11, ticks =634
Tick: PID 5,cpu : Odd ,ticks_run 12, ticks =634
Tick: PID 5,cpu : Odd ,ticks_run 13, ticks =634
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =635
Tick: PID 5,cpu : Odd ,ticks_run 14, ticks =635
YIELD: PID 6,
Tick: PID 5,cpu : Odd ,ticks_run 15, ticks =635
Tick: PID 5,cpu : Odd ,ticks_run 16, ticks =636
Tick: PID 4,cpu : Even ,ticks_run 1, ticks =637
Tick: PID 5,cpu : Odd ,ticks_run 17, ticks =637
Tick: PID 4,cpu : Even ,ticks_run 2, ticks =638
Tick: PID 5,cpu : Odd ,ticks_run 18, ticks =638
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =639
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =640
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =641
YIELD: PID 6,
Tick: PID 3,cpu : Even ,ticks_run 3, ticks =642
YIELD: PID 3,
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =644
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =645
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =646
YIELD: PID 6,
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =647
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =648
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =649
YIELD: PID 6,
Tick: PID 6,cpu : Even ,ticks_run 1, ticks =651
Tick: PID 6,cpu : Even ,ticks_run 2, ticks =652
Tick: PID 6,cpu : Even ,ticks_run 3, ticks =653
YIELD: PID 6,
--- Throughput Measurement ---
Tick: PID 3,cpu : Even ,ticks_run 1, ticks =655
Tick: PID 3,cpu : Even ,ticks_run 2, ticks =656
Total Ticks: 53
Completed Processes: 3
Throughput (Completed/Tick): 3 / 53
```